안녕하세요, 바흐티요르입니다 풀스택 AI 엔지니어입니다

React·TypeScript와 Node.js·Express를 중심으로 웹 서비스를 개발해 왔습니다. AI 기능을 사용자의 조작과 서비스 흐름에 연결하고, 요청 실패와 데이터 저장까지 함께 고려하며 개발합니다.

연락처

자기소개

React·TypeScript와 Node.js·Express를 중심으로 화면, 서버 API, 데이터베이스를 연결하는 풀스택 AI 엔지니어 바흐티요르입니다. 프리랜서 협업과 개인·팀 프로젝트를 통해 사용자 중심의 웹 서비스와 AI 기능을 개발해 왔습니다.

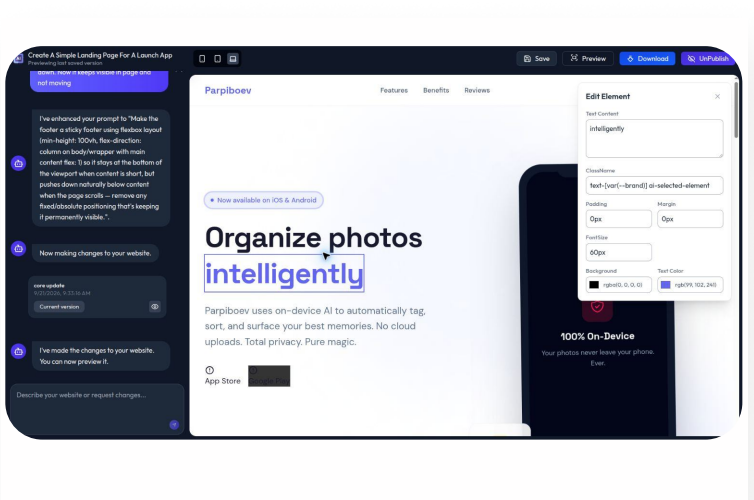
AI Web Builder에서는 생성 결과의 직접 편집과 저장, 요청 실패 시 크레딧 반환을 구현했습니다. Friendly에서는 서버·인증과 파인튜닝 데이터 준비·학습 실행, 챗봇 API·화면 연동을 담당했습니다. Dojang에서는 학교 공지·시간표·학업 기능과 리마인더·PWA를 개발했습니다.

사용자의 작업 흐름과 예외 상황을 함께 고려하며, 복잡한 처리를 작은 책임으로 나누고 팀과 협업해 기능을 완성합니다.

개인 웹사이트 bakhtiyor.me를 배포하고, 메타데이터·사이트맵 등 기술적 SEO와 Search Console 등록, GitHub Actions 검증을 구성했습니다. 프로젝트별 담당 업무와 구현 경험은 아래 문서에서 확인할 수 있습니다.

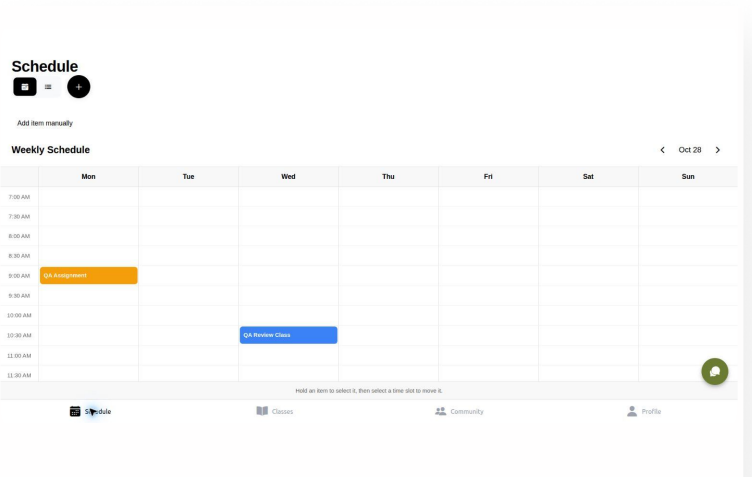
상세 개발 경험 보기 ↗

프로젝트 소개



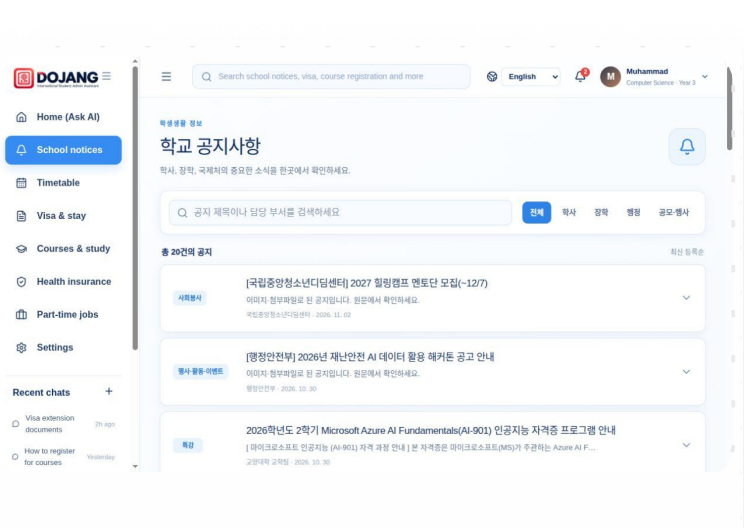
AI Web Builder

자연어로 웹사이트를 생성하고, 화면에서 직접 수정할 수 있는 AI 웹 제작 도구입니다. 텍스트·색상·간격을 직접 편집해 작은 수정마다 AI를 다시 기다리는 부담을 줄이고, 사용자가 만드는 재미와 주도감을 느끼도록 설계했습니다.



Friendly

시간표와 일정, 학습 자료, 강의 녹음, 과제 관리와 커뮤니티를 한곳에 모으고 AI 챗봇을 연결한 5인 팀 프로젝트입니다. 서버·인증, 학습 데이터 준비와 OpenAI 파인튜닝·챗봇 연동을 담당했으며, 캘린더 화면 구현과 Postman API 테스트에도 참여했습니다.



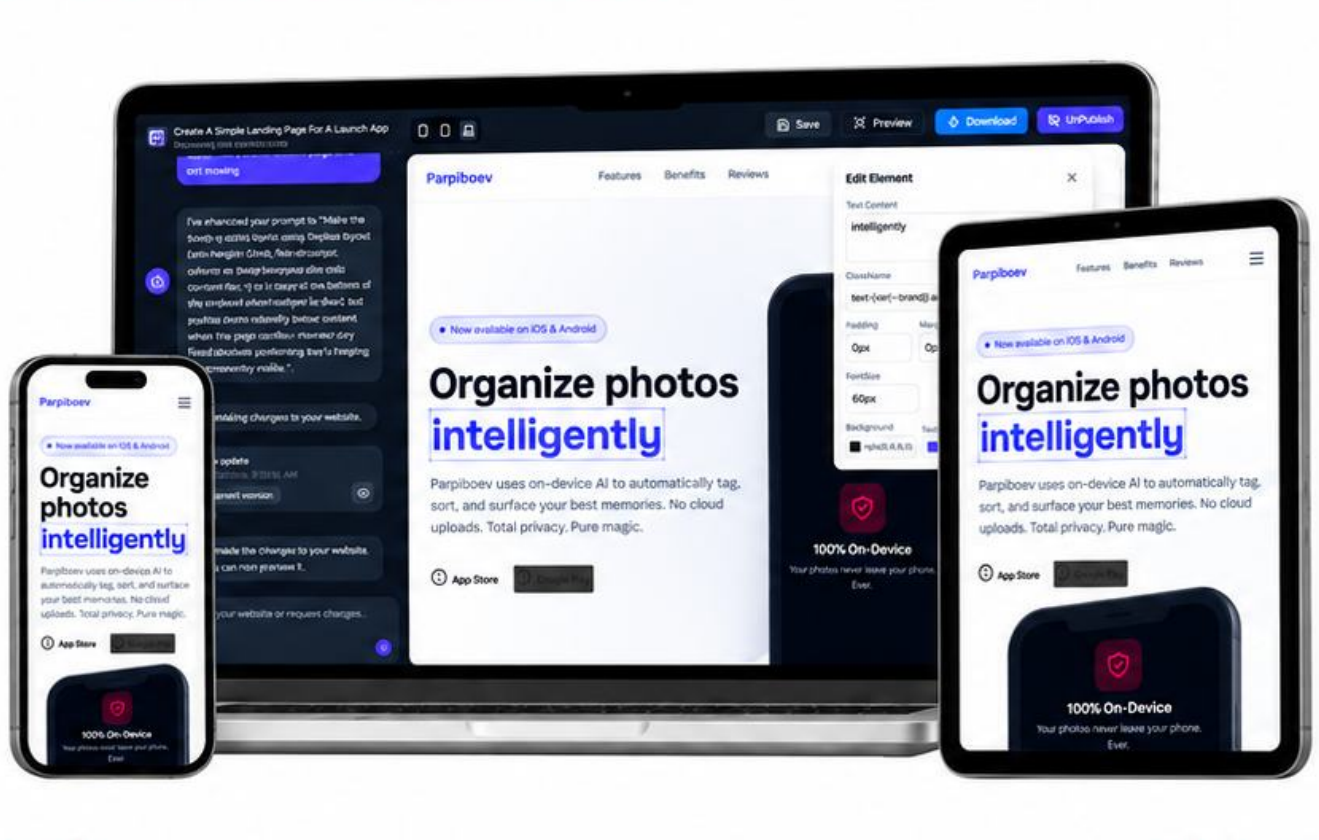
Dojang

한국에 거주하는 외국인 유학생을 위한 AI 행정·학업 지원 서비스로, 5인 팀으로 개발했습니다. 학생 설문조사를 진행하고 학교 공지, 시간표, 수강·학업 기능과 리마인더, PWA 기능을 구현했습니다.

01 / PERSONAL PROJECT

개인 프로젝트 · 베타

AI Web Builder



프로젝트 소개

AI 생성과 화면 직접 편집을 결합한 웹 제작 도구입니다. 자연어로 원하는 웹사이트를 설명하면 AI가 초안을 생성하고, 사용자가 미리보기 화면에서 결과물을 직접 다듬을 수 있는 웹 서비스입니다. 작은 수정마다 프롬프트를 다시 작성하고 기다리는 부담을 줄이기 위해 텍스트·색상·간격·글자 크기를 직접 편집하도록 구현했습니다. AI의 생성 기능에 사용자의 선택과 조작을 더해, 만드는 재미와 결과물에 대한 주도감을 느끼도록 설계했습니다.

주요 기능

- ✓ 자연어 기반 웹사이트 생성·수정
- ✓ 화면 요소 선택 및 텍스트·스타일 편집
- ✓ 반응형 미리보기 · 버전 저장 및 복원

기술 스택

Frontend	React · TypeScript · Tailwind CSS
Backend	Node.js · Express · PostgreSQL · Prisma
AI	OpenAI SDK → OpenRouter → NVIDIA Nemotron

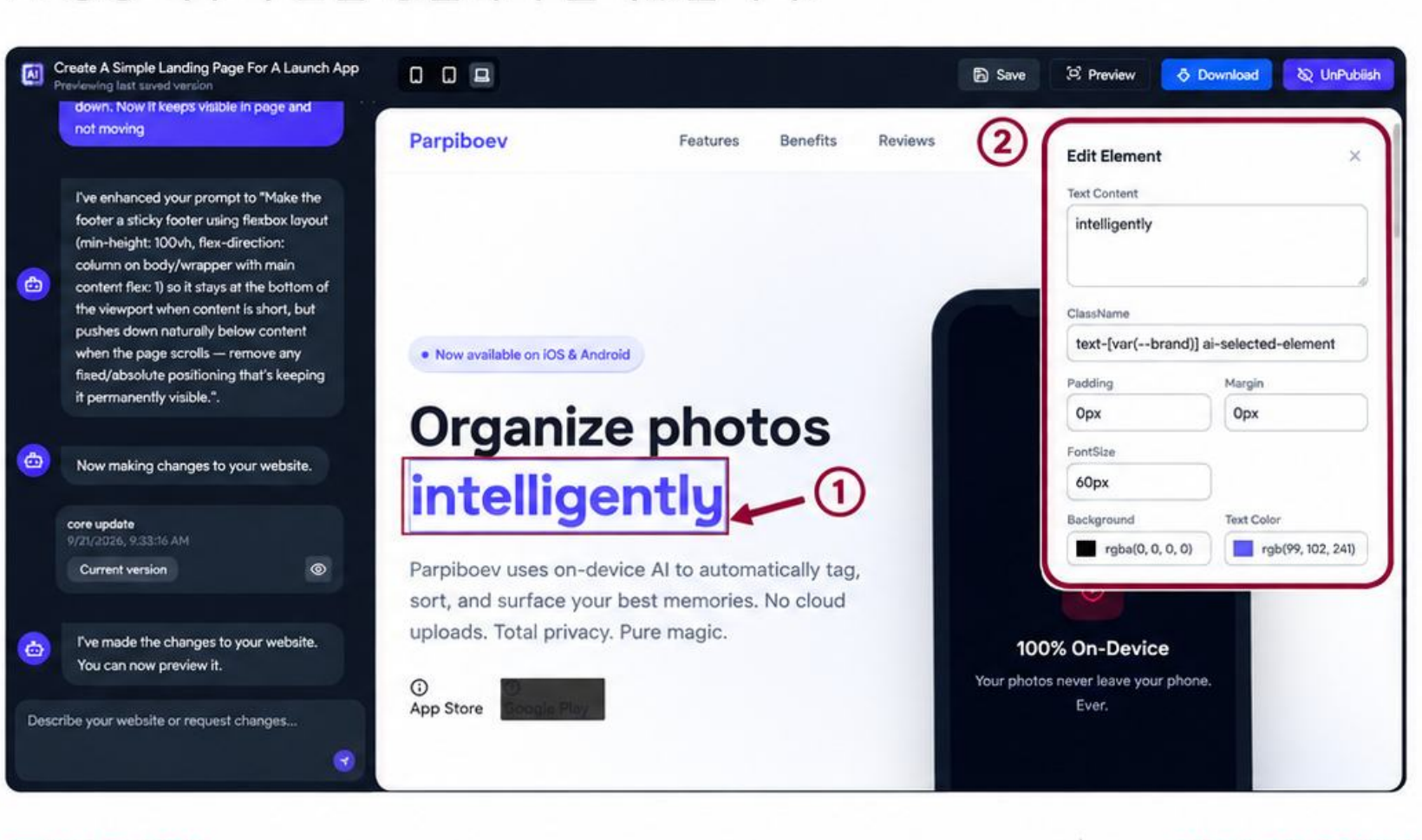
Better Auth 인증 · Stripe 테스트 모드 / 실제 과금은 향후 계획

구현 코드 · GitHub ↗

AI WEB BUILDER / 02

작은 수정은 화면에서 직접

AI 생성 이후의 편집 경험까지 설계했습니다.



문제

작은 수정도 프롬프트를 다시 쓰고 AI 응답을 기다려야 하는 부담이 있습니다.

설계 선택

AI 초안 생성과 직접 편집을 결합해 텍스트·색상·간격·글자 크기를 사용자가 조절하도록 구현했습니다.

사용 흐름

요소 선택 → 속성 편집 → 미리보기 확인 → 저장

구현 연결

요소 선택과 편집 패널을 연결하고 변경 내용을 미리보기에 반영합니다.

사용자 경험 의도

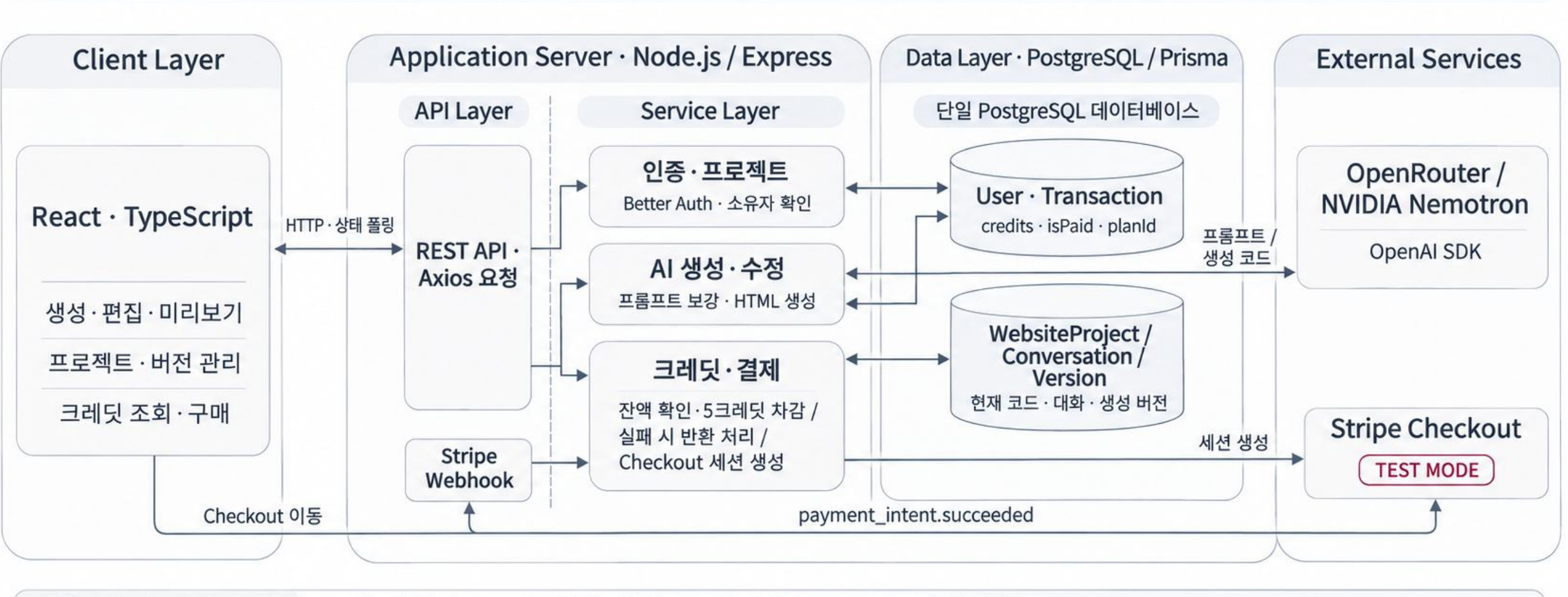
직접 만드는 재미와 결과물에 대한 주도감을 주고자 했습니다.

실제 화면 캡처 기반 · 저장 흐름은 코드에서 확인 · 사용성 개선 수치는 미측정

구현 코드 · GitHub ↗

AI Web Builder 시스템 아키텍처

웹 생성 · 직접 편집 · 크레딧 구매를 하나의 서비스로 연결



① 생성·수정 | 잔액 확인 → 5크레딧 차감 → AI 호출 → 코드·버전 저장

② 테스트 구매 | 거래 생성 → Stripe Checkout → 성공 웹훅 → 크레딧 충전

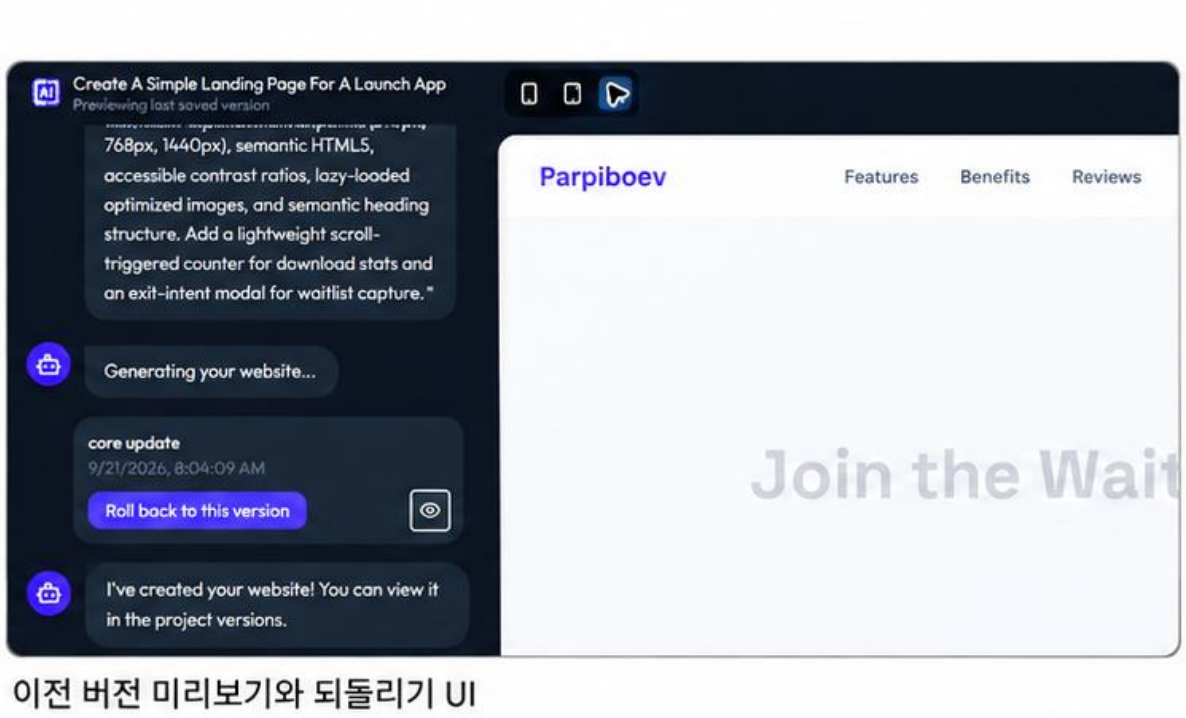
Stripe 테스트 모드 연동 · 실제 과금 검증은 향후 계획

구현 코드 · GitHub ↗

AI WEB BUILDER / 04

현재 코드와 생성 버전을 나누어 관리

AI 수정·직접 편집·되돌리기의 저장 방식을 구분했습니다.



이전 버전 미리보기와 되돌리기 UI

AI 수정	새 Version 생성 → current_code와 선택 버전 갱신
직접 편집 저장	current_code 갱신 → 선택 버전 해제 새 Version은 생성하지 않음
버전 되돌리기	소유자·프로젝트 내 버전 확인 → 저장 코드 복원 되돌리기 내용을 대화에 기록

데이터 관계도 (주요 엔티티)

User —1 → N— **WebsiteProject** (current_code · current_version_index) —1 → N— Conversation —1 → N— **Version** (code · description · timestamp)

현재 한계: 수동 편집은 별도 스냅샷이 없어 이전 버전 복원 시 덮어써질 수 있습니다.

구현 코드 · GitHub ↗

AI WEB BUILDER / 05 · CHALLENGES & LEARNING

예외 상황을 점검하고, 기능을 나누어 해결했습니다

01 실패한 요청의 크레딧 반환

문제

서버 연결 중단, 웹사이트 생성이 15분 이상 지연되는 상황, OpenRouter·모델 오류를 사용자 입장에서 반복 테스트했습니다. 요청 결과를 받지 못했는데 크레딧이 차감되는 상황을 점검했습니다.

해결

실패 처리 경로에 이미 차감된 5크레딧을 반환하는 로직을 추가했습니다. 정상적인 생성뿐 아니라 예외 상황에서도 사용자의 크레딧 처리를 함께 고려했습니다.

배운 점

성공하는 흐름만 확인하지 않고, 예상 밖의 사용자 시나리오로 오류 지점을 찾는 방법을 배웠습니다.

02 화면 요소 선택과 변경 내용 저장

문제

직접 편집 기능의 목표는 명확했지만, 화면 요소를 선택하고 수정한 내용을 DB에 저장하는 방법을 구현하는 데 약 일주일이 걸렸습니다.

해결

시니어 프론트엔드 개발자의 조언을 받아 마우스·호버 기반 요소 선택과 조작 방식을 익혔습니다. 저장 처리는 백엔드의 별도 함수로 분리해 수정한 코드를 DB에 반영했습니다.

배운 점

혼자 해결하기 어려운 부분은 전문가에게 조언을 구하고, 선택·편집·저장처럼 역할별로 기능을 나누는 중요성을 배웠습니다.

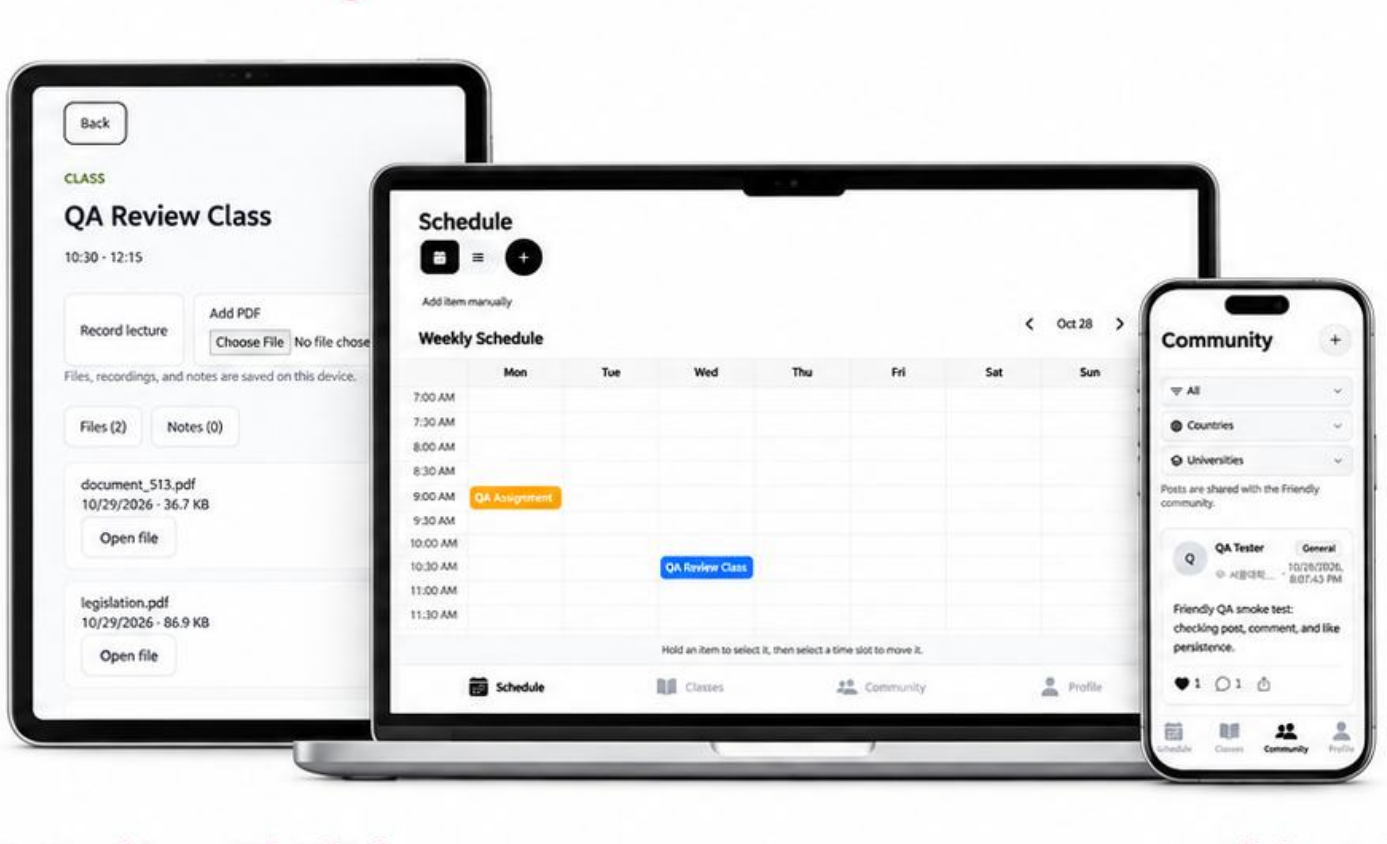
다음 단계 프롬프트 품질 개선 — 생성·수정 결과를 비교하며 지시문 개선 | Stripe 실결제 전환 — 실제 결제·웹훅·크레딧 반영 검증

구현 코드 · GitHub ↗

FRIENDLY / 01 · TEAM PROJECT

5인 팀 프로젝트 · 웹 구현

Friendly



프로젝트 소개

시간표와 일정, 학습 자료, 강의 녹음, 과제와 커뮤니티를 한곳에 모은 학생 생활 지원 서비스입니다. 유학생이 학교생활에 필요한 정보를 관리하고 AI 챗봇으로 학업·생활 관련 질문을 할 수 있도록 5인 팀으로 개발했습니다.

나의 역할

Express 서버·인증, 학습 데이터 작성·정제와 OpenAI 파인튜닝·챗봇 연동을 담당했습니다. 캘린더·공통 UI 구현과 Postman API 테스트에도 참여했습니다.

주요 기능 · 팀 전체

- ✓ 일정·캘린더 및 과제 상태 관리
- ✓ 강의 녹음 · PDF·노트 관리
- ✓ AI 챗봇 · 커뮤니티

기술 스택

Frontend	React Native · Expo · TypeScript
Backend	Node.js · Express · Axios
Auth / Data	Firebase Auth · Firestore
AI	OpenAI API · 파인튜닝

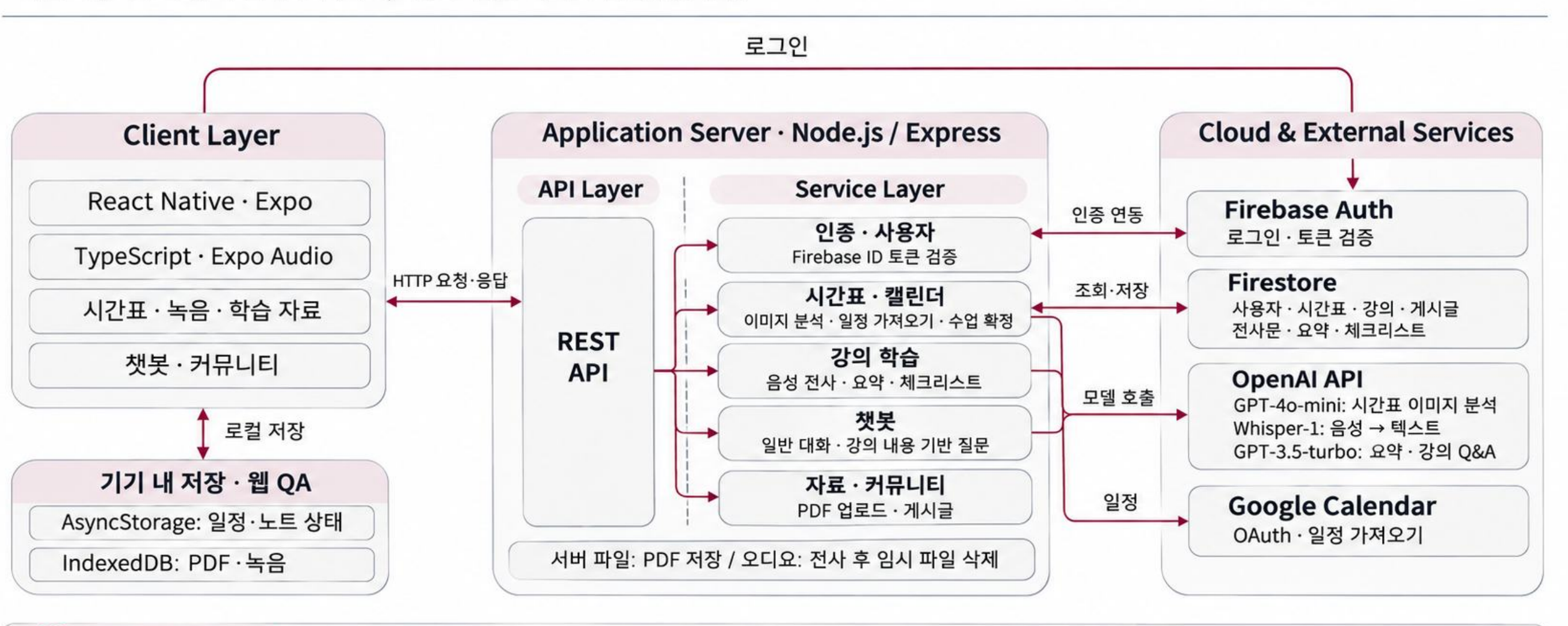
기기 화면은 웹 UI 기반 목업 · AI 실호출 검증은 API 크레딧 부족으로 보류

구현 코드 · GitHub ↗

FRIENDLY / 02 · SYSTEM ARCHITECTURE

Friendly 시스템 아키텍처

시간표 등록부터 강의 전사·요약까지, 학습 흐름을 하나로 연결했습니다.



① 시간표 등록 | 이미지 분석 또는 Calendar 가져오기 → 확인·수정 → 저장 → 강의 항목 생성

② 강의 학습 | 녹음 종료 → Whisper 전사 → 요약·체크리스트 생성 → Firestore 저장

팀 통합 코드 기준 · 웹 QA의 로컬 저장 경로는 별도 표시 · 공공데이터포털: 대학 기본정보용 설정 확인, 호출 경로 확인 필요

구현 코드 · GitHub ↗

FRIENDLY / 03 · MY CONTRIBUTION

서버와 챗봇을 사용자 화면으로 연결했습니다

5인 팀 / 개인 기여

담당 범위: Express 서버 · Firebase 인증 · 챗봇 API·화면 · 일정 UI

제가 연결한 서비스 흐름

01	인증 연결	회원가입·로그인 → Firebase ID 토큰 검증
02	서버 구성	Express 라우트 분리 · 인증·사용자·일정·챗봇 연결
03	AI 요청	챗봇 메시지 → 서버 → OpenAI API
04	화면 연결	질문 입력 · 메시지 표시 · 전송 중 상태

구현에서 맡은 부분

01 **서버의 연결 지점 구성**
인증·사용자·일정·챗봇 라우트를 Express 서버에 연결했습니다.

02 **학습·생활 도움 챗봇**
한국의 신입생과 유학생을 돕는 지시문을 구성하고, 모델 호출을 화면과 연결했습니다.

03 **일정과 공통 UI**
주간 캘린더·목록 화면과 튜토리얼·공통 UI 구현에 참여했습니다.

코드 근거 · 인증 라우트

```
POST /api/auth/signup
POST /api/auth/login
POST /api/auth/verify
```

routes/auth.js · 회원가입 / 로그인 / 토큰 검증

구현 이력

81fd3c5	인증 엔드포인트
75e09dc	백엔드 챗봇·Postman 테스트
43e8965	서버 구성 수정

개인 기여는 확인된 담당 범위로 한정하고, 팀 전체의 강의 전사·요약 기능과 구분했습니다.

구현 코드 · GitHub ↗

FRIENDLY / 04 · AI INTEGRATION & VERIFICATION

학습 데이터 준비부터 모델 연동까지 담당했습니다

개인 기여 / 모델 평가는 제외

데이터 작성·정제 → OpenAI 파인튜닝 실행 → 챗봇 모델 연동

01 데이터 준비	02 파인튜닝 실행	03 모델 연동	04 API 확인
다국어 대화 예제 작성·정제	OpenAI 학습 작업 수행	MODEL_ID로 모델 지정	Postman 요청·응답 테스트
system · user · assistant	준비한 학습 데이터 사용	서버에서 API 호출	챗봇 API 동작 확인

모델을 코드에 고정하지 않고 설정으로 연결

환경변수 MODEL_ID를 읽어 OpenAI 요청의 model 값으로 전달했습니다. 학습한 모델을 챗봇 호출 경로에 연결하는 부분을 담당했습니다.

코드 발췌 · 모델 선택

```
const model = process.env.MODEL_ID;
model: model,
```

routes/AI_chatBot.js · c5d172c

학습 실행 기록과 모델 성능 개선 수치는 제시하지 않습니다.

Postman으로 API 연결 확인

챗봇 API에 직접 요청을 보내 응답을 확인했습니다. 프론트엔드와 연결하기 전, 서버의 요청·응답 흐름을 확인하는 데 활용했습니다.

확인 가능한 작업 근거

학습 데이터	messages 형식 JSONL
모델 연동	c5d172c
Postman 기록	75e09dc 커밋 메시지

이번 경험에서 다룬 범위 | 학습 데이터 준비 · 파인튜닝 실행 · 모델 연동 · API 테스트

모델 결과 평가는 담당하지 않았으며, 현재 배포의 파인튜닝 모델 사용 여부와 별도로 설명합니다.

구현 코드 · GitHub ↗

FRIENDLY / 05 · CHALLENGES & LESSONS

모델의 답변 범위와 협업 방식을 다듬은 경험

파인튜닝과 5인 팀 개발에서 마주한 문제, 대응, 배운 점

01 모델의 역할과 답변 범위 정하기

문제

파인튜닝을 위해 질문·답변 중심의 JSONL 데이터를 약 2,000개 준비했습니다. 하지만 학습 과정에서 모델의 말투가 의도와 달라지거나, Friendly와 관련 없는 주제로 답변하는 경우가 있었습니다. 학습 데이터를 준비하는 것만으로 서비스에 맞는 답변 범위가 정해지지는 않았습니다.

대응

모델에 instructions를 추가해 역할과 답변할 주제를 명시했습니다. 프로젝트의 범위를 벗어난 질문에는 답변을 제한하도록 지시하며, 학습 데이터와 별도로 챗봇이 따라야 할 응답 규칙을 정리했습니다.

배운 점

학습 예제를 만드는 작업과 서비스의 응답 규칙을 설계하는 작업을 함께 고려해야 한다는 점을 배웠습니다. Instructions는 모델을 원하는 방향으로 유도하는 방법이며, 모든 범위 이탈을 막는 기술적 차단과는 구분해야 한다고 생각합니다.

02 작업 브랜치와 통합 과정 나누기

문제

5명이 하나의 프로젝트를 함께 개발하면서 서로 작성한 코드를 push하고 합치는 과정에서 충돌이 발생했습니다. 각자 기능을 구현하는 것뿐 아니라, 여러 사람의 변경 사항을 어떤 순서와 방식으로 통합할지도 정해야 했습니다.

대응

별도의 작업 브랜치를 만들어 그 안에서 개발하고 push한 뒤, 팀장이 메인 브랜치에 병합하는 방식으로 작업 흐름을 나눴습니다. 개인이 작업하는 공간과 팀 전체의 코드를 통합하는 단계를 분리했습니다.

배운 점

브랜치를 나누는 것이 충돌 자체를 없애지는 않지만, 각자의 변경 사항을 구분하고 통합 절차를 정리하는 데 필요한 방법이라는 점을 배웠습니다. 이후 협업에서는 기능 분담과 함께 브랜치 사용 방식과 통합 담당도 먼저 합의하려고 합니다.

개인 경험을 바탕으로 정리 · 학습 데이터: JSONL 1,991개 확인

구현 코드 · GitHub ↗

DOJANG / 01 · TEAM PROJECT

Dojang

5인 팀 프로젝트 · MVP

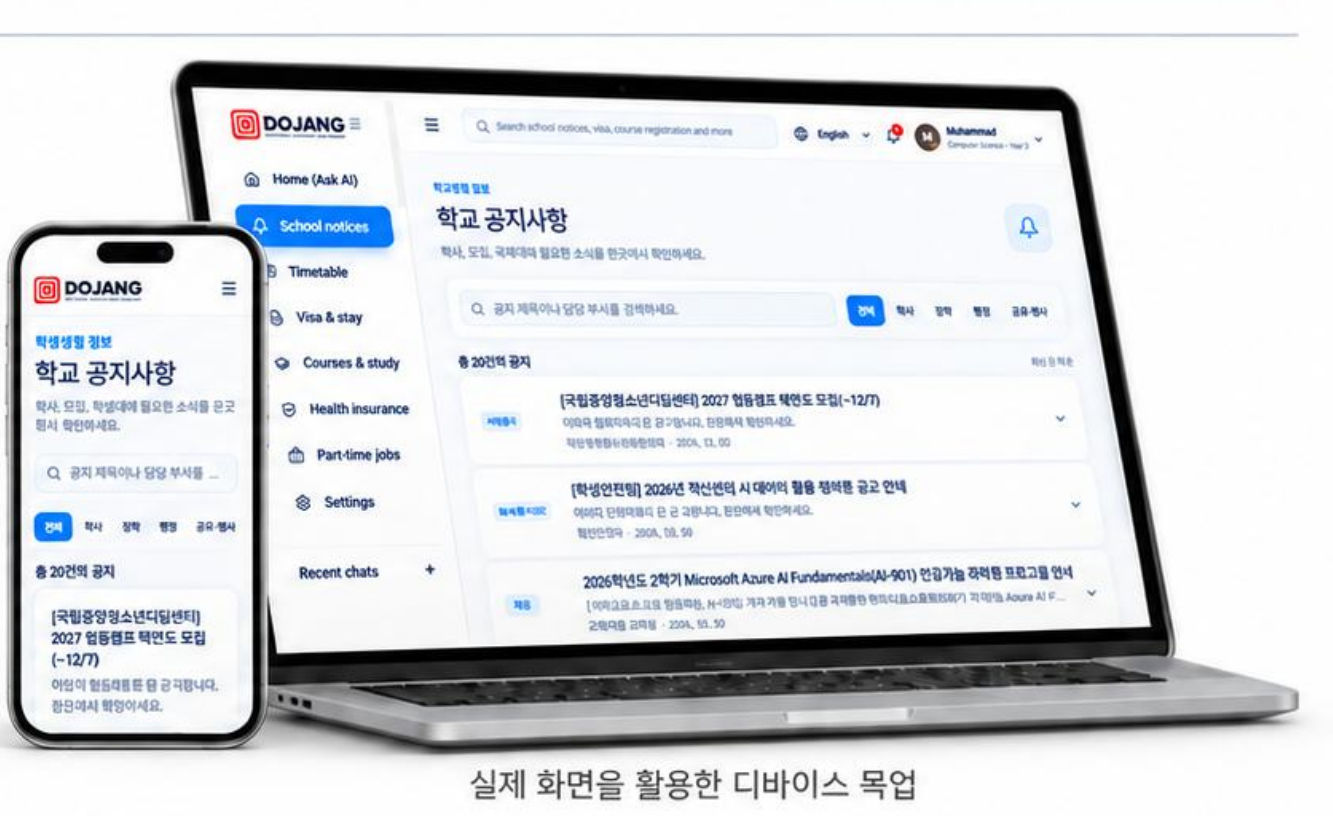
프로젝트 소개

국민대학교 외국인 유학생을 위한 AI 행정·학업 지원 플랫폼입니다. 학교 공지, 시간표, 학업 상태와 비자·건강보험 관련 일정을 한곳에서 확인하도록 구성했습니다.

AI 도우미는 규정 검색과 서비스별 도구를 활용해 정보를 조회하고, 확인 가능한 출처와 함께 답변합니다.

담당 역할

학생 설문조사 참여, 학교 공지 수집 API·화면, 시간표, 학업 상태 계산·화면, 알림·체크리스트, PWA 기능을 담당했습니다.



실제 화면을 활용한 디바이스 목업

주요 기능

- ✓ 4개 언어 AI 답변 · 규정 검색
- ✓ 학교 공지 검색 · 분류
- ✓ 주간 시간표 · 학점·GPA 상태
- ✓ 비자·보험 일정 안내 · 알림·체크리스트
- ✓ 예시 공고 조건 매칭 · PWA·오프라인 화면

기술 스택

Frontend	Next.js · React · TypeScript
UI	Tailwind CSS · CSS Modules
Backend	Node.js · Express · TypeScript · Cheerio
AI · 검색	Anthropic SDK · Tool calling · BM25
브라우저	localStorage · Service Worker

MVP · 시간표·성적·구인 공고는 데모 데이터 · 학교 공지는 공개 웹사이트 수집 · 알림은 앱 내 표시

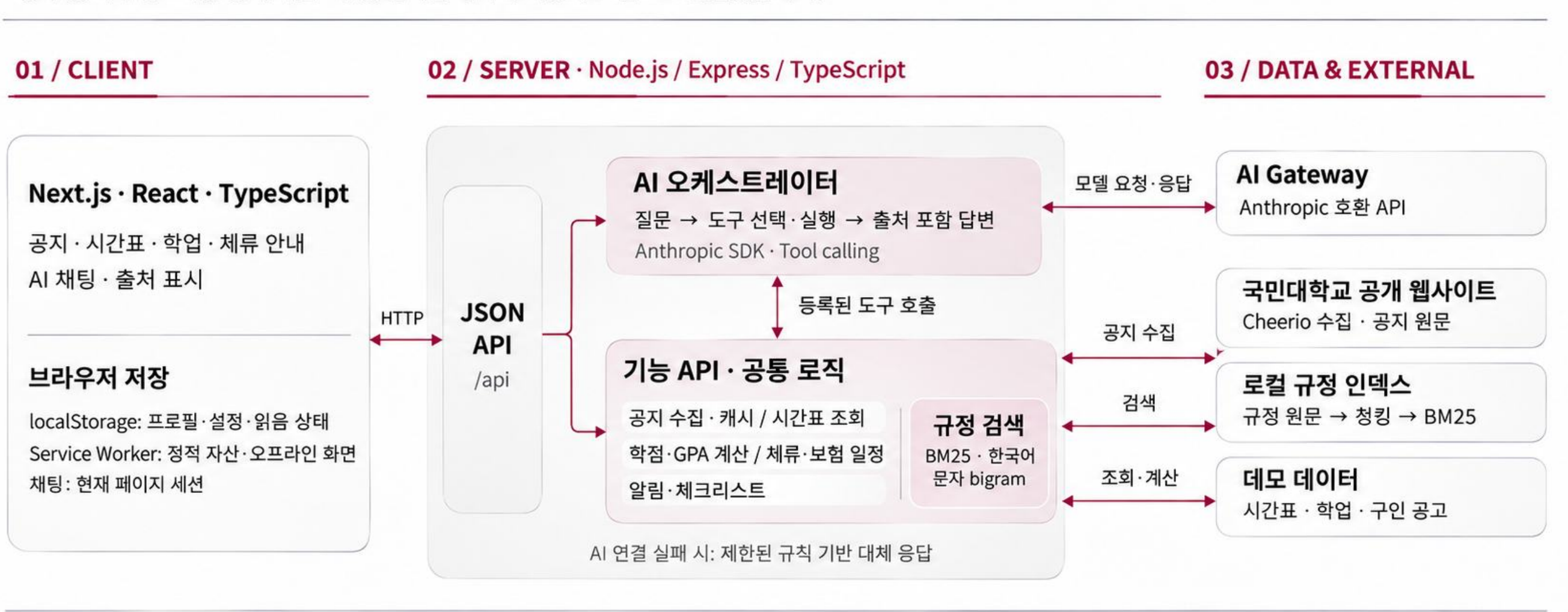
구현 코드 · GitHub ↗ 서비스 데모 ↗

DOJANG / 02 · SYSTEM ARCHITECTURE

팀 전체 구조 · MVP

정보 조회와 AI 안내를 연결한 서비스 구조

공개 공지 수집 · 학업 상태 계산 · 규정 검색을 하나의 사용자 흐름으로 연결했습니다.


01 / CLIENT
Next.js · React · TypeScript
공지 · 시간표 · 학업 · 체류 안내
AI 채팅 · 출처 표시
브라우저 저장
localStorage: 프로필·설정·읽음 상태
Service Worker: 정적 자산·오프라인 화면
채팅: 현재 페이지 세션
HTTP
02 / SERVER · Node.js / Express / TypeScript
JSON API
/api
AI 오케스트레이터
질문 → 도구 선택·실행 → 출처 포함 답변
Anthropic SDK · Tool calling
등록된 도구 호출
기능 API · 공통 로직
공지 수집 · 캐시 / 시간표 조회
학점·GPA 계산 / 체류·보험 일정
알림·체크리스트
규정 검색
BM25 · 한국어 문자 bigram
AI 연결 실패 시: 제한된 규칙 기반 대체 응답
모델 요청·응답
공지 수집
검색
조회·계산
03 / DATA & EXTERNAL
AI Gateway
Anthropic 호환 API
국민대학교 공개 웹사이트
Cheerio 수집 · 공지 원문
로컬 규정 인덱스
규정 원문 → 청킹 → BM25
데모 데이터
시간표 · 학업 · 구인 공고

일반 조회 | 화면 → 기능 API → 조회·계산 → 결과 표시

AI 안내 | 질문 → 오케스트레이터 → 도구 실행 → 근거와 함께 답변

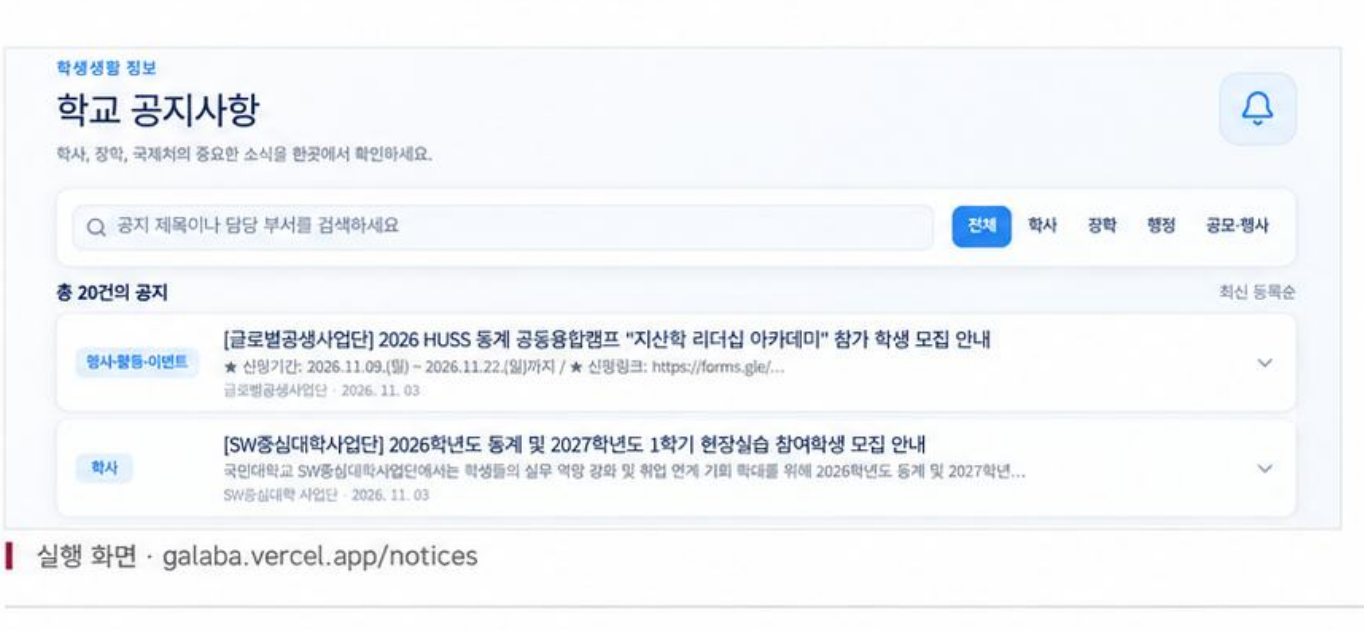
프로필은 브라우저에 저장 · 시간표·학업·구인 공고는 데모 · 캘린더는 일정 생성 링크 제공

구현 코드 · GitHub ↗ 서비스 데모 ↗

DOJANG / 03 · MY CONTRIBUTION

학교 공지와 시간표를 탐색 가능한 화면으로 구현했습니다

공개 웹사이트의 공지 수집부터 주간 수업 정보 표시까지 담당했습니다.



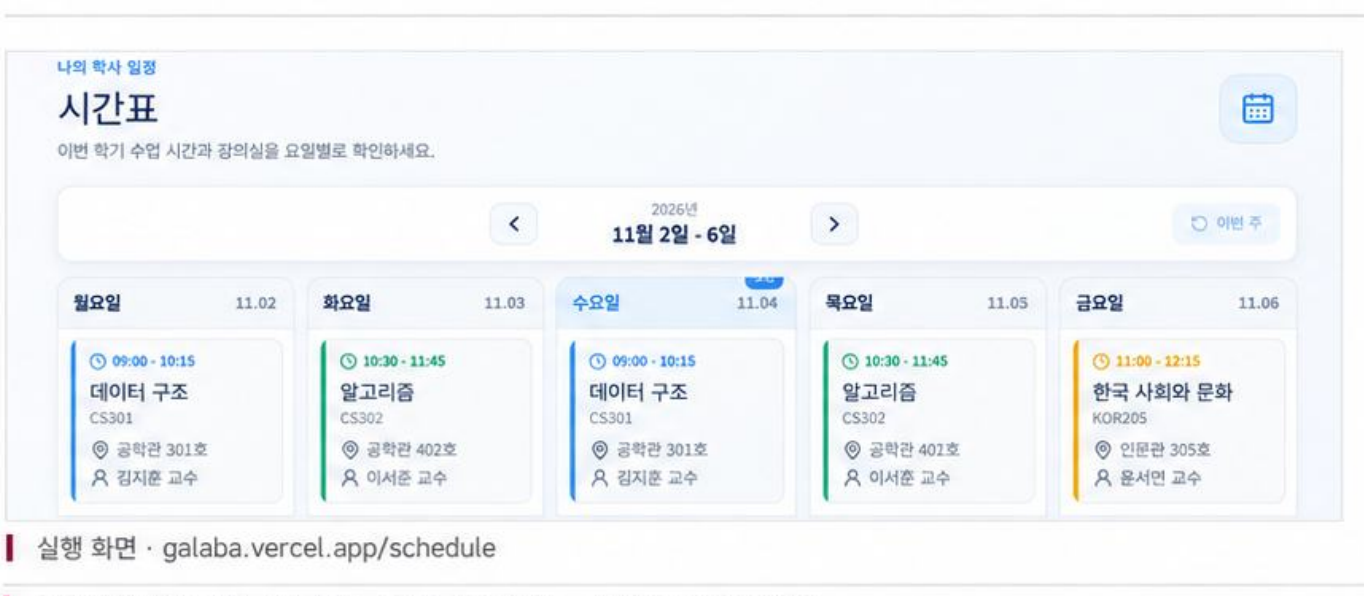
실행 화면 · galaba.vercel.app/notices

01 학교 공지 수집 API · 화면

국민대학교 공개 웹사이트의 공지 목록과 본문을 수집하고, 제목·날짜·담당 부서·원문 링크를 화면에 연결했습니다.

- ✓ 검색·분류로 필요한 공지 탐색
- ✓ 목록에서 요약 확인 · 원문 이동
- ✓ 로딩·빈 결과·오류 상태 표시

`GET /api/notices · backend/src/notices/crawler.ts`



실행 화면 · galaba.vercel.app/schedule

02 주간 시간표 API · 화면

요일별 수업을 주간 화면에 배치하고, 과목·시간·교수·강의실을 함께 확인하도록 구성했습니다.

- ✓ 이전·다음 주 이동과 오늘 강조
- ✓ 모바일에서는 요일별 탭 표시
- ✓ API의 수업 정보를 화면에 연결

`GET /api/schedule · frontend/src/app/schedule/page.tsx`

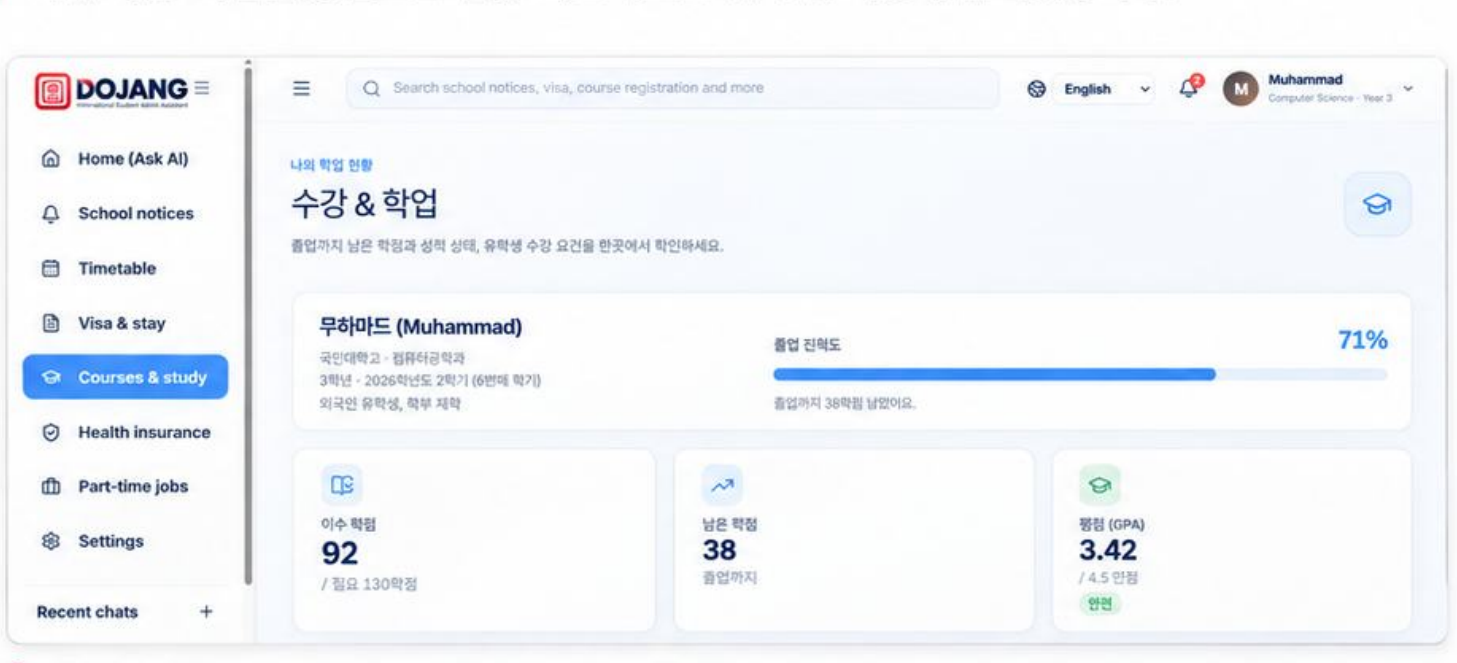
담당 범위: 공지 수집 API·화면, 시간표 API·화면 · 시간표는 데모 데이터

구현 코드 · GitHub ↗ 서비스 데모 ↗

DOJANG / 04 · MY CONTRIBUTION

학업 상태를 계산하고, 확인할 일과 연결했습니다

학업 계산·화면을 중심으로 알림·체크리스트와 PWA 기능에 참여했습니다.



실행 화면 · galaba.vercel.app/academics · 데모 학업 정보

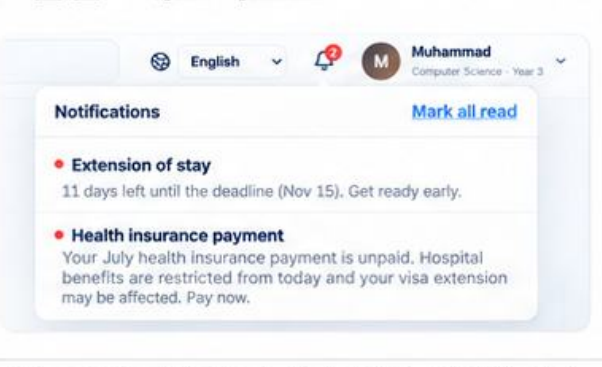
01 학업 상태 계산 · 화면

이수·필요·남은 학점과 졸업 진척도를 계산하고, GPA 상태를 화면에 표시했습니다. 계산 로직과 기준값을 분리해 관리하도록 구성했습니다.

- ✓ 학점 진행률 · 남은 학점 표시
- ✓ GPA 상태 · 수강 요건 안내
- ✓ 로딩·오류·재시도 상태 구성

Evidence `GET /api/academics · academics/compute.ts`

02 알림 · 체크리스트



기한별 안내와 준비 항목을 확인하고, 읽음 상태를 관리하는 앱 내 알림 기능을 담당했습니다.

외부 메시지·푸시 발송 기능은 포함하지 않습니다.

03 PWA · 오프라인 화면

Manifest → Service Worker → Offline 화면

설치용 manifest와 서비스 워커를 구성하고, 정적 자산 캐시와 오프라인 대체 화면을 제공했습니다.

API 조회와 AI 응답은 인터넷 연결이 필요합니다.

Evidence `frontend/public/sw.js · app/manifest.ts`

팀 전체 AI·비자 계산 기능과 개인 기여를 구분했습니다 · 학업 데이터는 MVP 시연용

구현 코드 · GitHub ↗ 서비스 데모 ↗ 학업 기능 커밋 ↗

바흐티요르

전공 지식과 도메인 지식을 바탕으로, 요구사항 이해부터 구현·검증·개선까지 연결하는 사용자 중심의 앱과 웹을 개발하고자 합니다.

풀스택 AI 엔지니어

LinkedIn ↗
+82-10-8437-6644
bakhtiyor@kookmin.ac.kr
GitHub ↗
bakhtiyor.me ↗